

MODULE 2

Installation and Setup

Choose a deployment, install or provision MongoDB, connect with mongosh and Compass, and prove the environment with a write and a read.

**Choose a
deployment**

Install and run

Connect

First commands

**Secure and
troubleshoot**

LEARNING OBJECTIVES

What you will be able to do by the end of this module.



BY THE END OF THIS MODULE, YOU CAN

1. Choose local, container, self-managed or cloud
2. Explain mongod, mongosh, Compass and drivers
3. Install or provision MongoDB and start it
4. Read and build a connection string
5. Create training_store and run CRUD checks
6. Protect credentials and diagnose failures



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHERE

should MongoDB run for this course?

HOW

do I install and start mongod?

HOW

does a client find the server?

WHAT

proves the environment really works?

HOW

do I keep passwords out of sight?

WHAT

do I check when it won't connect?

KEY TAKEAWAY A setup is finished only when a ping, a write and a read all succeed.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 **mongod**

The MongoDB server process: it listens on a port, stores the data files and answers every request.

2 **mongosh**

The MongoDB Shell: a command-line client that sends commands to mongod and prints the results.

3 **MongoDB Compass**

MongoDB's graphical client for browsing and editing the same data that mongosh sees.

4 **MongoDB Atlas**

MongoDB's managed cloud service: the vendor runs, backs up and scales the cluster.

5 **Connection string**

A URI such as `mongodb://localhost:27017` that tells a client where and how to connect.

6 **localhost:27017**

The default address of a local mongod: this machine, port 27017.

7 **Authentication vs authorization**

Authentication proves who you are; authorization (roles) decides what you may do.

8 **Ping**

`db.runCommand({ ping: 1 })` returns `ok: 1` when the server answers — reachability, not write access.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

CLI

Command-Line Interface

GUI

Graphical User Interface

SRV

Service (DNS SRV record)

CRUD

Create, Read, Update, Delete

IP

Internet Protocol

TLS

Transport Layer Security

DB

Database

MSI

Microsoft Installer (Windows
Installer package)

URI

Uniform Resource Identifier

DNS

Domain Name System

OS

Operating System

VM

Virtual Machine

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 1 TO MODULE 2

The ideas are in place — now we build the environment.

MODULE 1 GAVE US

- ✓ NoSQL is a family of data models
- ✓ Documents, collections and BSON
- ✓ mongod serves mongosh and Compass
- ✓ The training_store sample database
- ✓ Exercise 1.3 on the instructor's server

NOW Run your own MongoDB — and prove that it works.



THE PATH THROUGH MODULE 2

- 1 Choose where MongoDB runs
- 2 Install, configure and start mongod
- 3 Connect with a connection string
- 4 Create training_store; write and read
- 5 Protect credentials; troubleshoot
- 6 Prove it: Exercises 2.1–2.4, Lab 1

Part 1

Part 2

Part 3

Part 4

Part 5

Checkpoints

KEY TAKEAWAY Module 1 explained what MongoDB is; Module 2 gives every learner a running, verified copy of it.

PART 1: CHOOSING A DEPLOYMENT

Before installing anything, decide where MongoDB should run.



SO FAR

- ✓ Module 1 explained documents and mongod
- ✓ training_store is our running example

NEXT QUESTION

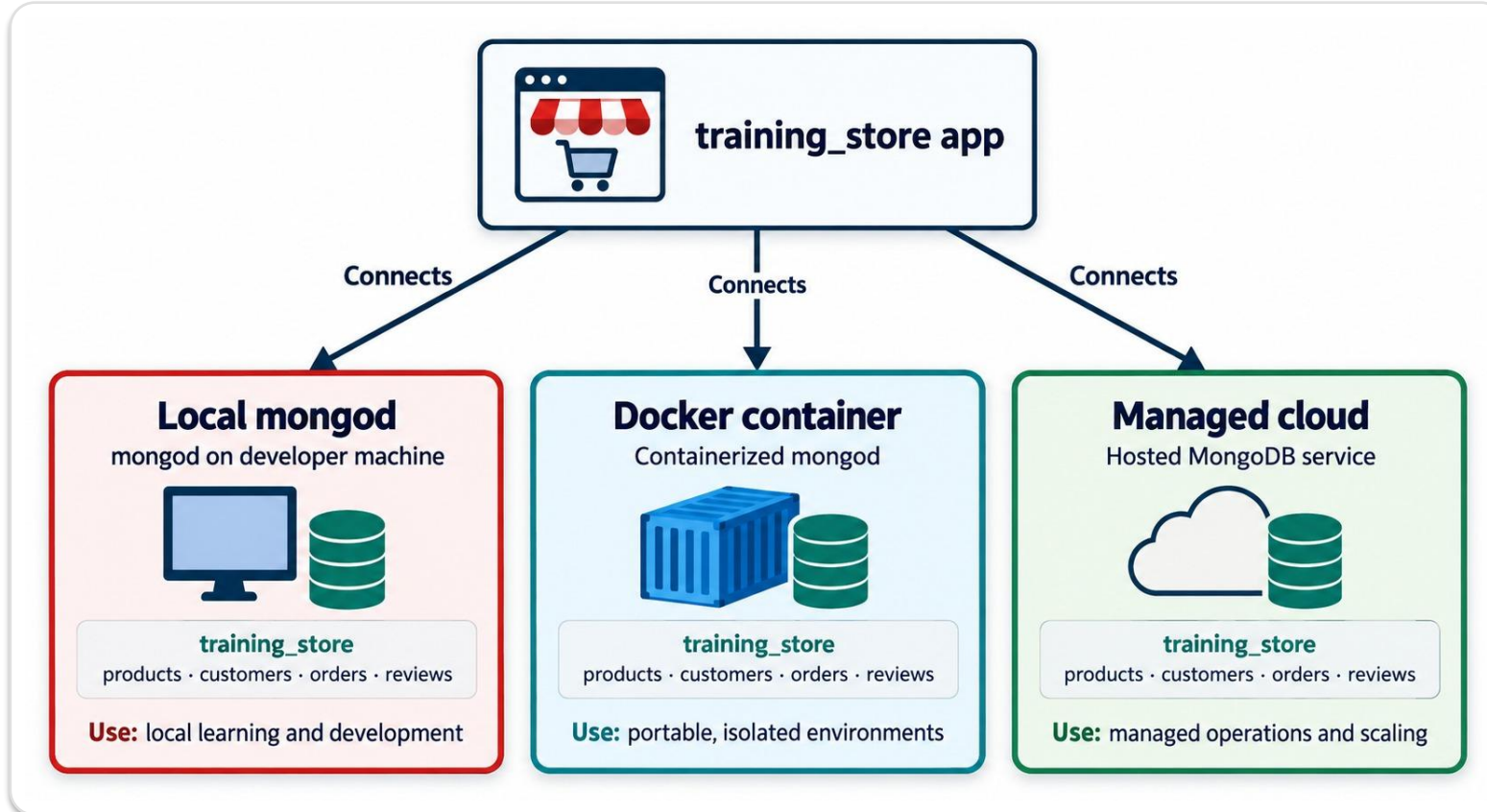
Where should MongoDB run — on my laptop, in a container, on a server, or in the cloud?

IN THIS PART

- 1 Four deployment options
- 2 Local Community Edition
- 3 Containerized MongoDB
- 4 Managed cloud: MongoDB Atlas
- 5 Local versus managed cloud
- 6 Choosing an option

MONGODB DEPLOYMENT OPTIONS

The same training_store database — three places to run it.



Local mongod

Community Edition on your own machine — learning and development.

Docker container

Portable and isolated, easy to remove — tests and demos.

Managed cloud

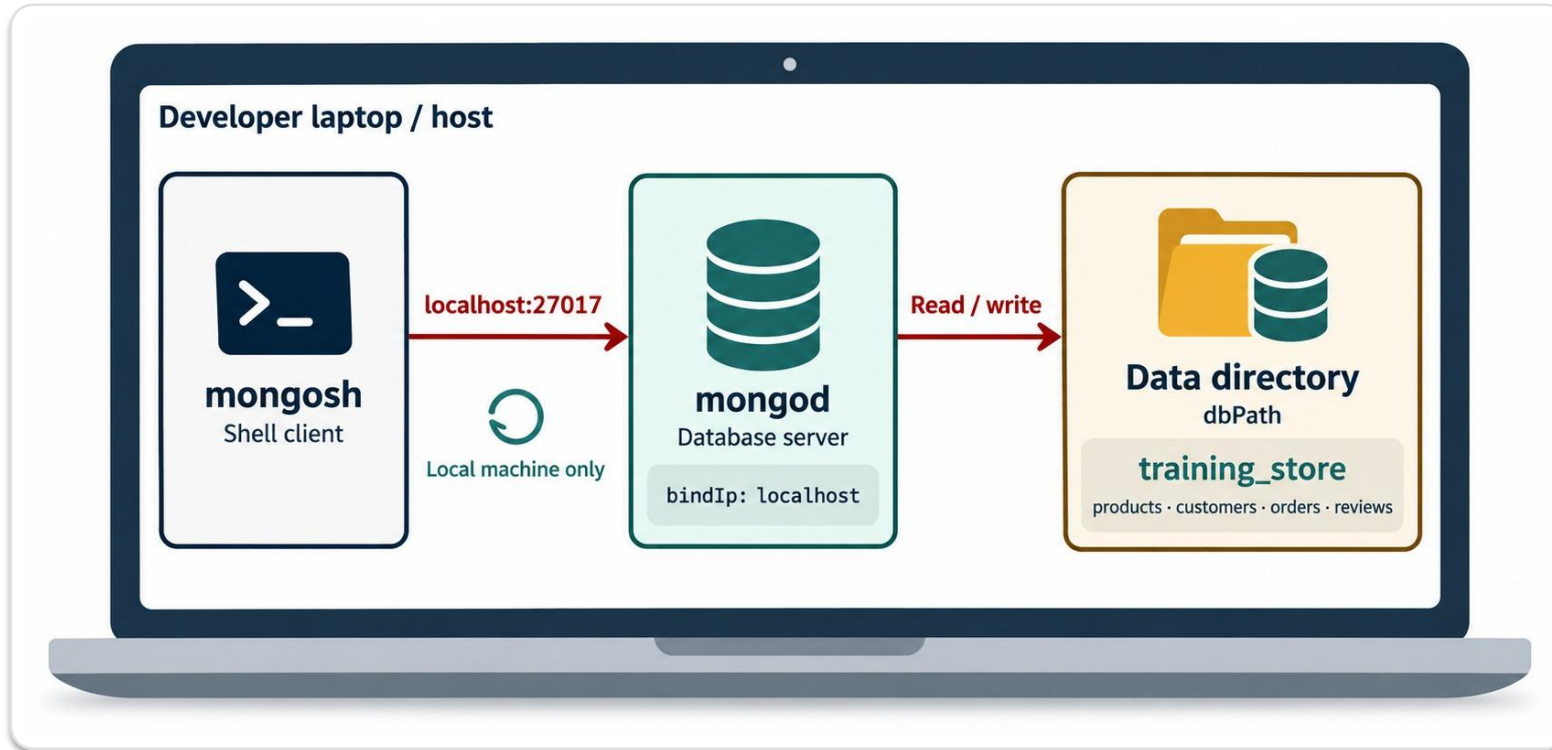
MongoDB Atlas runs the cluster — teams and production.

FOURTH OPTION A self-managed server or VM — see the decision flow.

KEY TAKEAWAY Choose the deployment by who should run the server — the database and the commands stay the same.

LOCAL DEPLOYMENT: MONGODB COMMUNITY EDITION

mongosh, mongod and the data directory on one machine.



Community Edition

The free server, installed from MongoDB's official packages.

localhost:27017

The default address; bindIp: localhost keeps it private.

dbPath

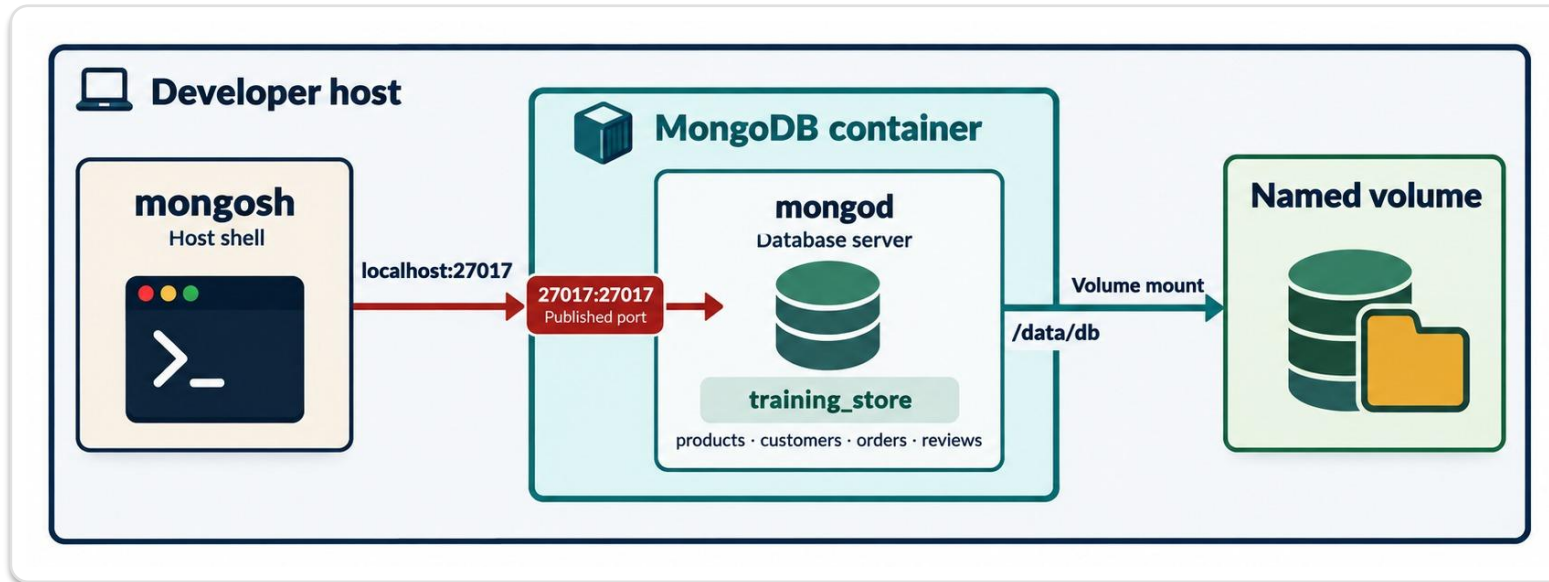
The data directory where mongod stores training_store.

AUTH IS OFF BY DEFAULT Fine on localhost only — never expose it.

KEY TAKEAWAY A local server is private by default — keep bindIp on localhost while authentication is off.

CONTAINERIZED MONGODB

mongod in a container, a published port and a named volume.



Start it (PowerShell)

```
docker run -d --name mongo `
  -p 127.0.0.1:27017:27017 `
  -v mongo-data:/data/db `
  mongodb/mongodb-community-server
```

Published port

Host 27017 → container 27017; 127.0.0.1 keeps it on this machine.

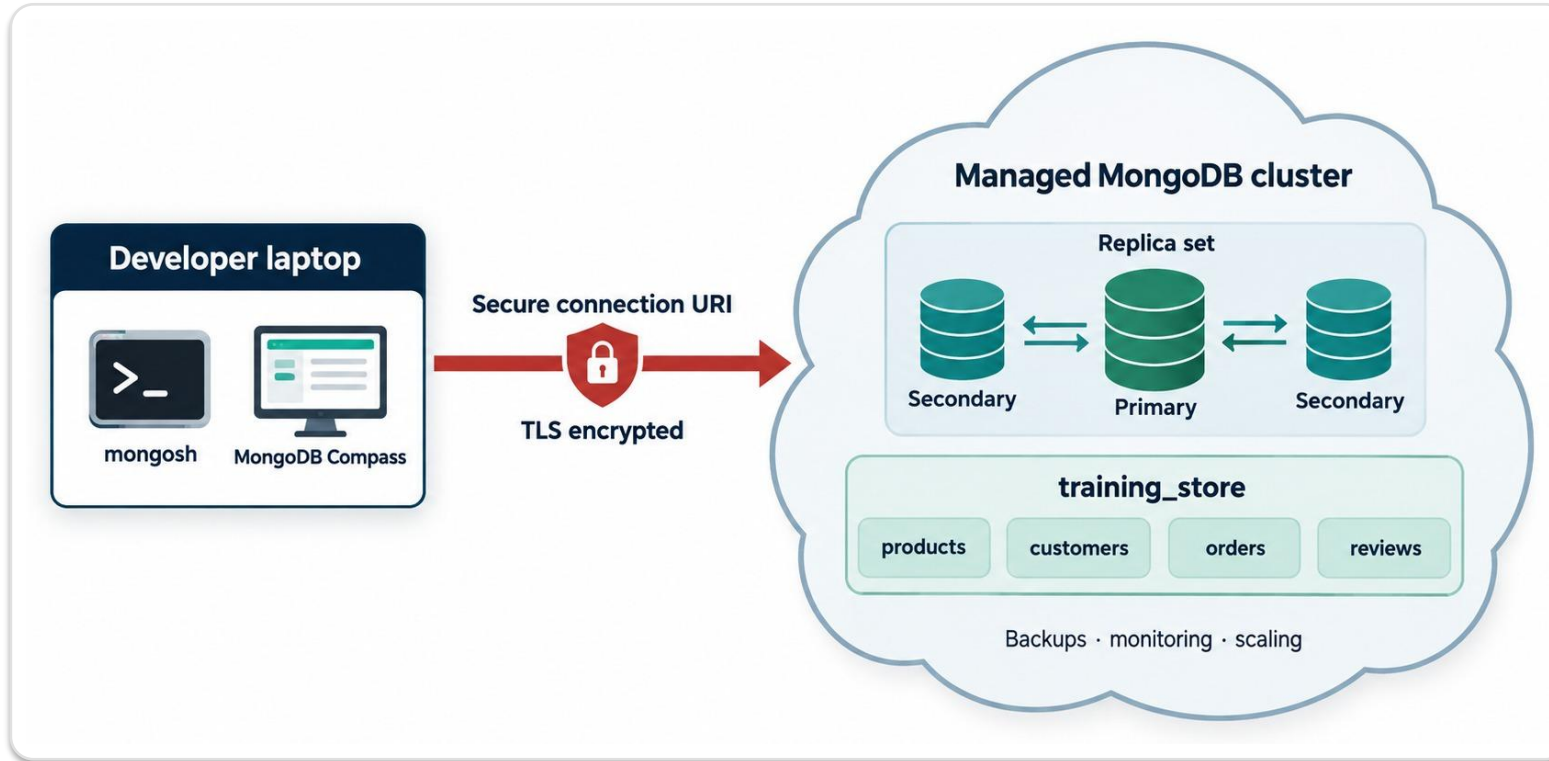
Named volume

mongo-data outlives the container — so does training_store.

KEY TAKEAWAY A container is quick to create and remove — mount a volume so the data survives.

MANAGED CLOUD: MONGODB ATLAS

A managed replica set, reached through a secure connection URI.



MongoDB Atlas

MongoDB's managed cloud: servers, patches, backups and scaling.

Replica set included

One primary and two secondaries from the start.

TLS required

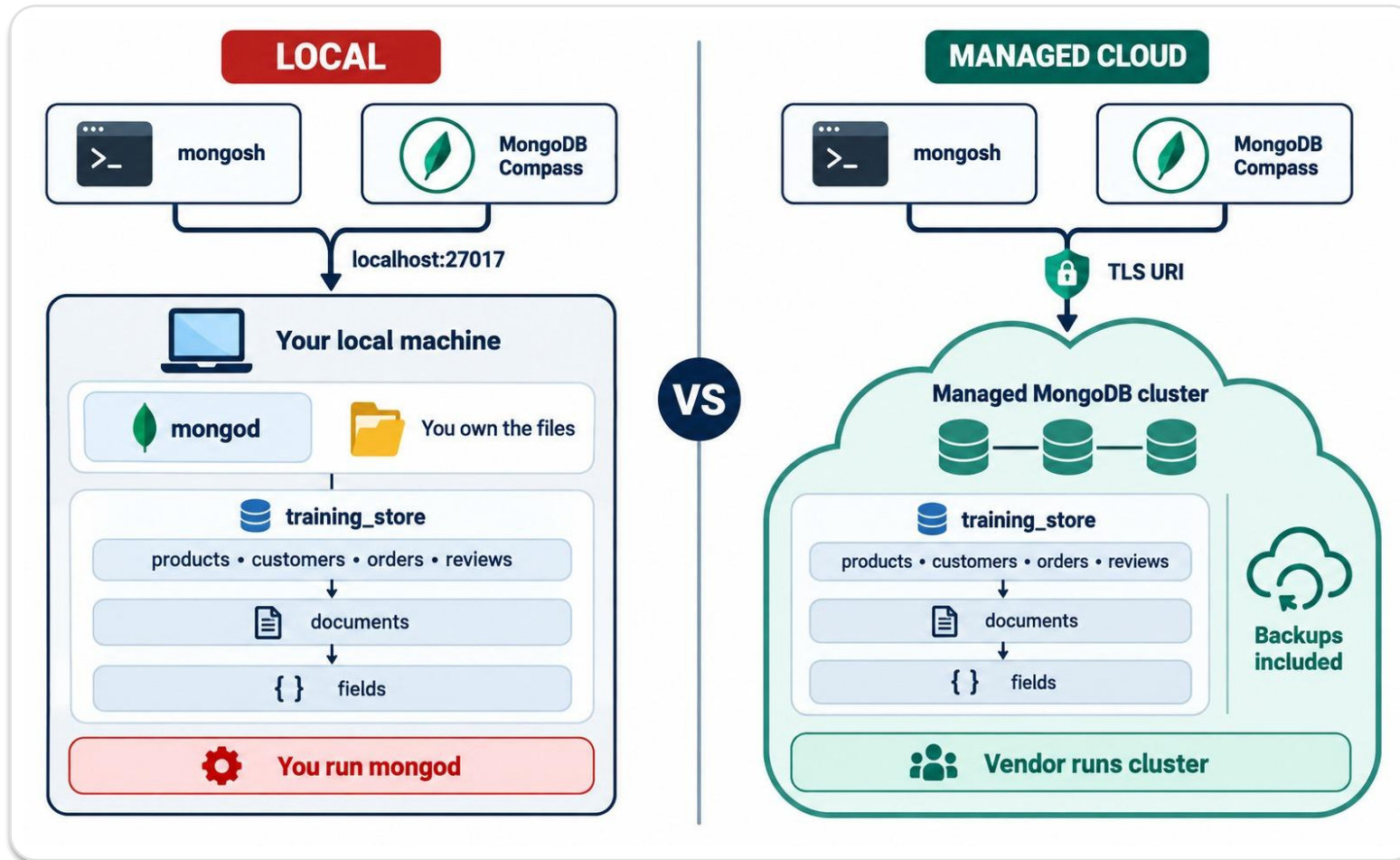
Clients connect with an encrypted connection URI.

YOU STILL OWN Database users, the IP access list and the secret URI.

KEY TAKEAWAY Atlas runs the servers for you — you still decide who may connect, and from where.

LOCAL VS. MANAGED CLOUD

You run mongod — or the vendor runs the cluster.



Internet

Local: not after installation · Atlas: always.

Who runs it

Local: you run mongod · Atlas: the vendor runs the cluster.

Address

Local: localhost:27017 · Atlas: a TLS connection URI.

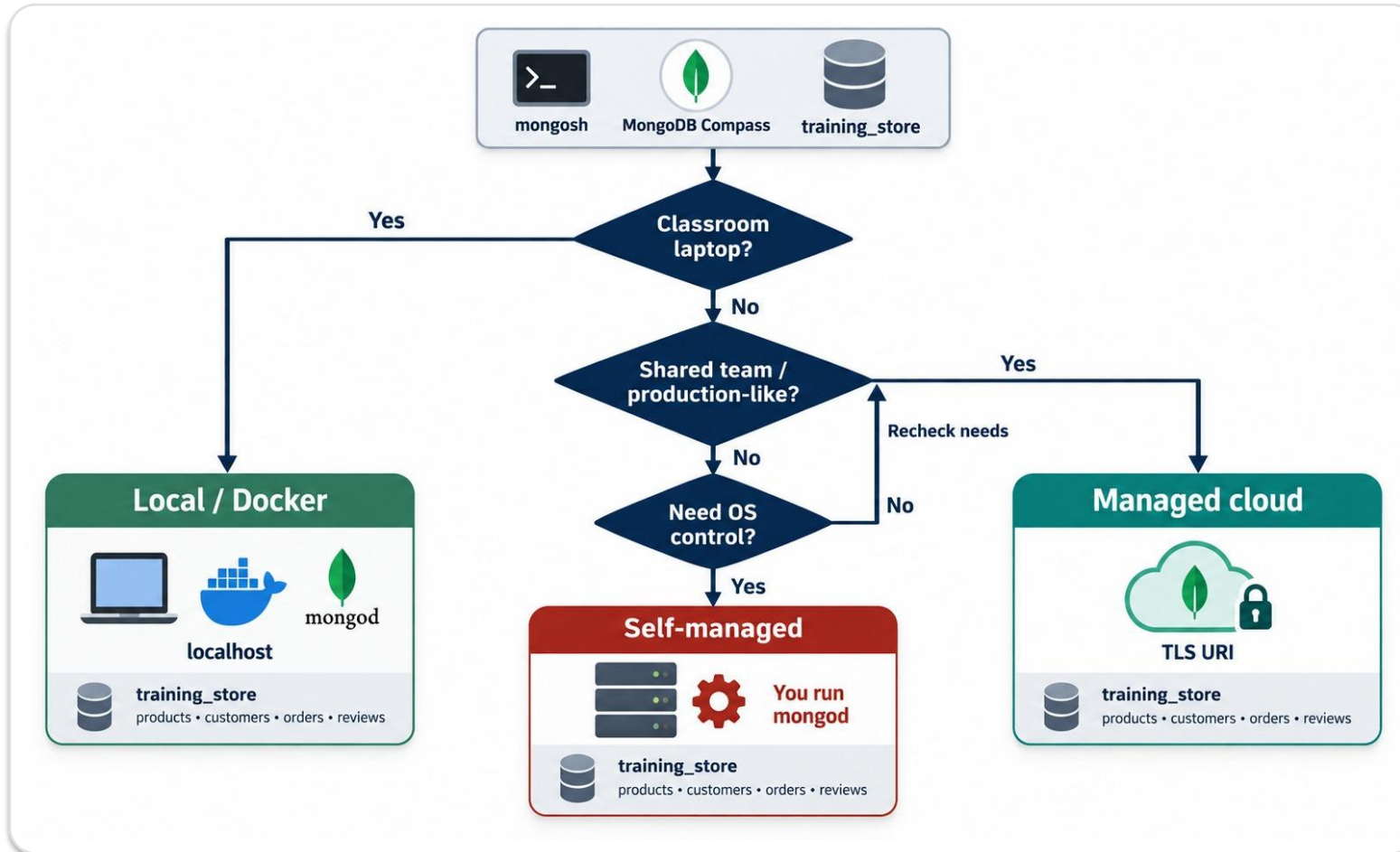
Best use

Local: learning, local development · Atlas: team and cloud projects.

KEY TAKEAWAY Same documents, same commands — local gives you control, the cloud takes the operations work.

CHOOSING A DEPLOYMENT OPTION

Three questions lead to local, self-managed or managed cloud.



Classroom laptop?

Local or Docker on localhost.

Shared team or production-like?

Managed cloud with a TLS URI.

Need control of the OS?

Self-managed server or VM — you run mongod.

EXERCISE 2.1 You'll justify six choices like these.

KEY TAKEAWAY Start from the constraint — offline, disposable, shared or controlled — then pick the option that fits it.

PART 2: INSTALLING AND RUNNING MONGODB

Install the server and the shell, configure them and start the service.



SO FAR

- ✓ Four deployment options, one database
- ✓ Local for learning; Atlas for teams
- ✓ The operating responsibility is what changes

NEXT QUESTION

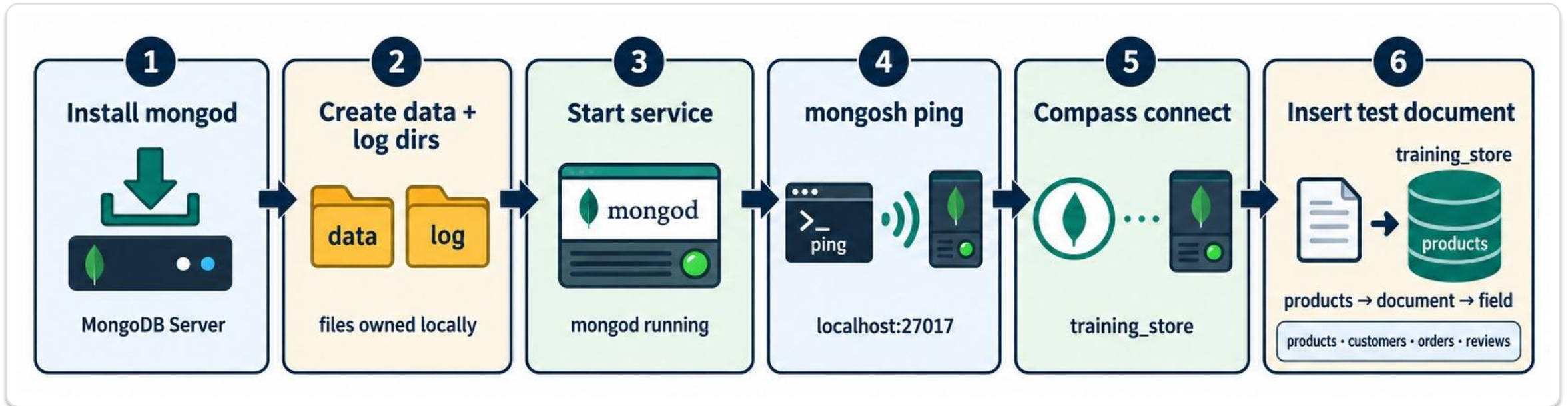
How do I install MongoDB, configure it and start it — or provision it in the cloud?

IN THIS PART

- 1 The installation workflow
- 2 mongod versus mongosh
- 3 Installing on Windows, macOS and Linux
- 4 The configuration file
- 5 Starting and stopping MongoDB
- 6 Provisioning in Atlas

THE MONGODB INSTALLATION WORKFLOW

Six steps from download to a first test document.



Server side

Install mongod, check the data and log directories, start the service.

Client side

Ping from mongosh, connect Compass, insert a test document.

What you install

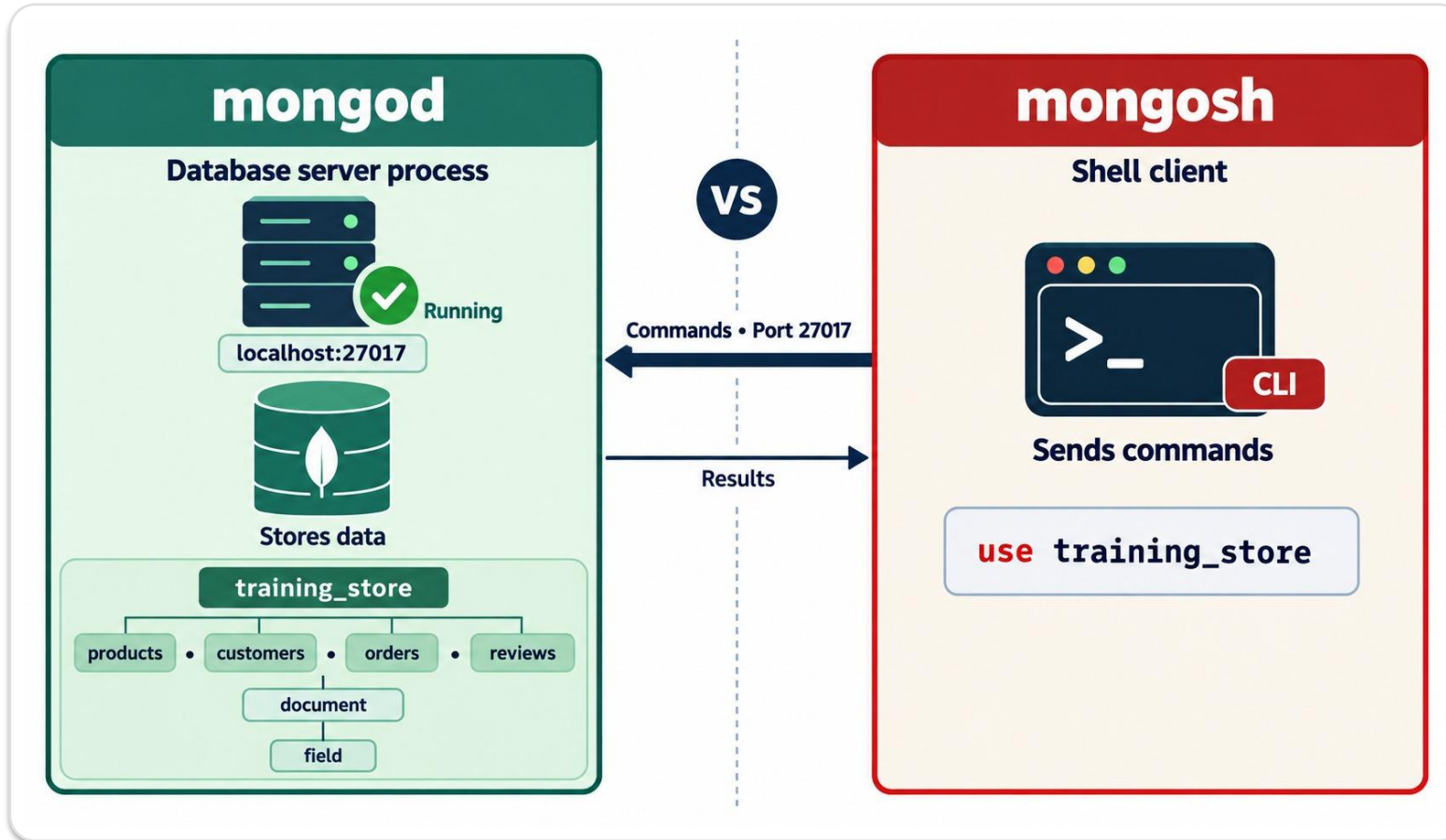
Community Server, mongosh, optional Compass and Database Tools.

OFFICIAL DOCS Follow MongoDB's install page for your OS and version.

KEY TAKEAWAY Installing is only half the job — the last three steps prove the server works.

MONGOD VS. MONGOSH

The server that stores the data and the shell that sends commands.



mongod

The server: stores the data and listens on port 27017.

mongosh

The shell: sends commands and prints the results.

Separate programs

Closing mongosh never stops the database.

NO SERVER, NO SHELL If mongod isn't running, mongosh can't connect.

KEY TAKEAWAY mongod is the database; mongosh is only a window onto it.

INSTALLING MONGODB ON WINDOWS, MACOS AND LINUX

Use MongoDB's official packages for your operating system.

Windows (PowerShell)

```
# MSI installer: Complete,  
# run as a Windows service,  
# optionally add Compass.  
# Add mongosh if not included.  
Get-Service MongoDB  
mongosh --version
```

macOS (Homebrew)

```
brew tap mongodb/brew  
brew install mongodb-community  
brew services start mongodb-community  
mongod --version  
mongosh --version
```

Linux (MongoDB's repository)

```
# add MongoDB's repository for  
# your distribution, then:  
sudo systemctl start mongod  
sudo systemctl status mongod  
sudo systemctl enable mongod
```

Windows

The MSI installs mongod as the MongoDB service.

macOS

The formula comes from MongoDB's own Homebrew tap.

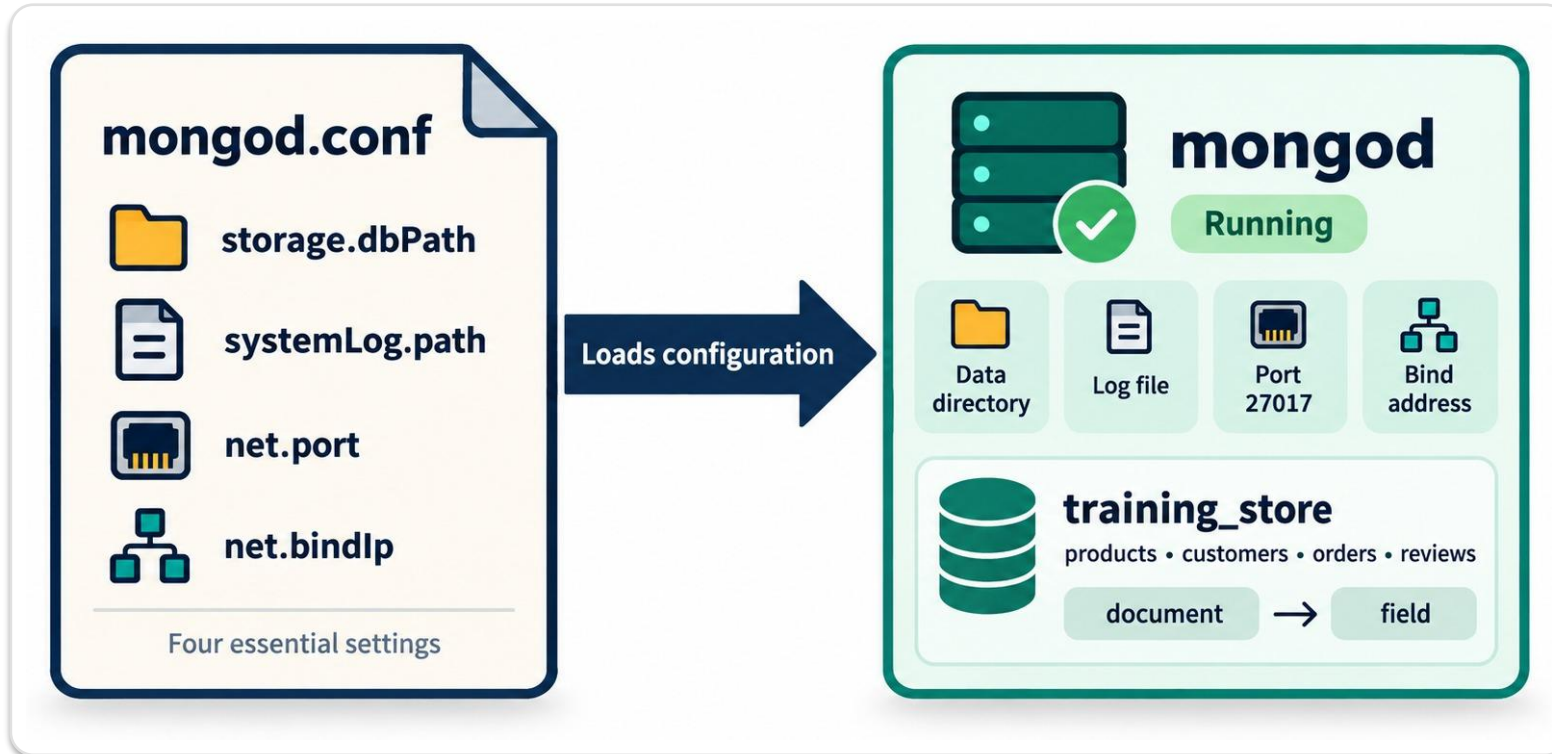
Linux

Use MongoDB's repository, not the distribution's default package.

KEY TAKEAWAY Install from MongoDB's official source, then confirm both the service and the shell before moving on.

THE MONGODB CONFIGURATION FILE

Four settings mongod loads when it starts.



mongod.conf (Linux example)

```
storage:
  dbPath: /var/lib/mongodb
systemLog:
  destination: file
  path: /var/log/mongodb/mongod.log
net:
  port: 27017
  bindIp: 127.0.0.1
```

bindIp 127.0.0.1

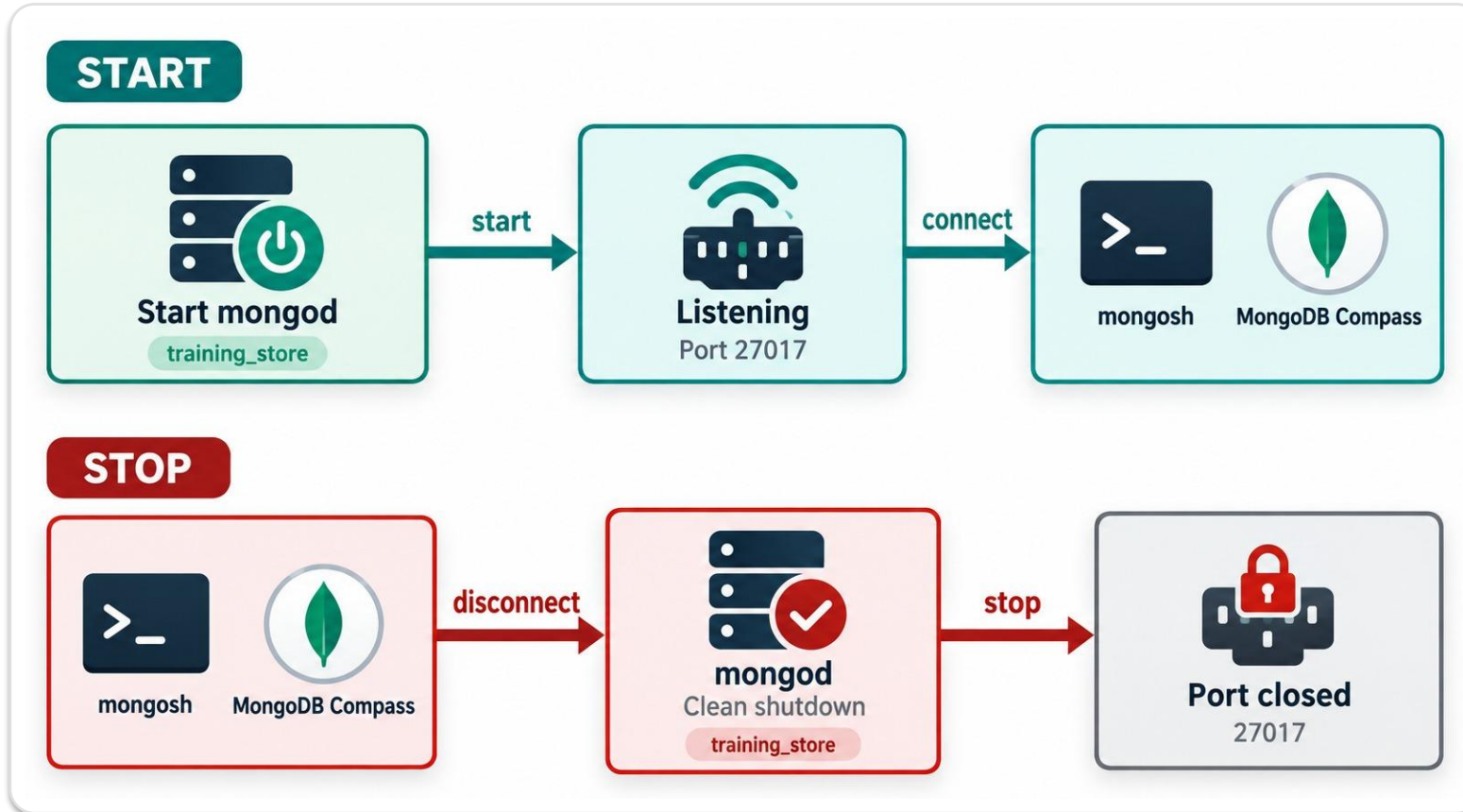
This machine only — keep it for learning.

WARNING 0.0.0.0 exposes mongod to every network — only with auth, TLS and a firewall.

KEY TAKEAWAY The config file decides where data and logs go and who can reach the port — read it first when something breaks.

STARTING AND STOPPING MONGODB

Start, listen, connect — then disconnect, shut down cleanly, close the port.



Windows (admin PowerShell)

```
Start-Service MongoDB
Stop-Service MongoDB
```

macOS

```
brew services start mongodb-community
brew services stop mongodb-community
```

Linux

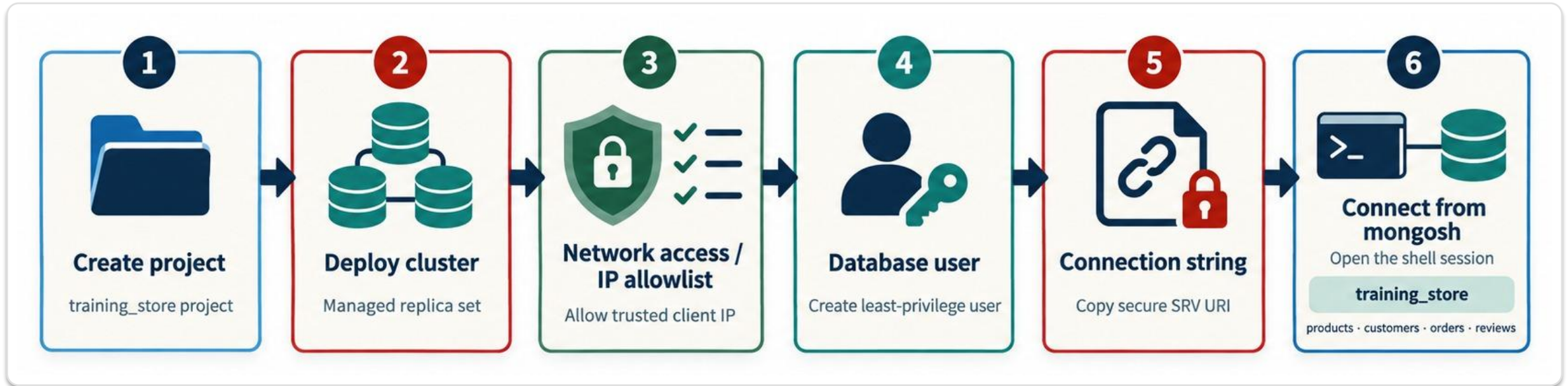
```
sudo systemctl start mongod
sudo systemctl stop mongod
```

CLEAN SHUTDOWN Use the service manager — don't kill the process.

KEY TAKEAWAY Start and stop mongod through its service manager, so every shutdown is clean.

PROVISIONING MONGODB ATLAS

Project, cluster, network access, database user, connection string, connect.



Project and cluster

Pick a free or paid tier, a provider and a region.

IP access list

Allow only the client addresses that need access.

Database user

Least privilege — not your Atlas login.

Connection string

Copy the SRV URI from the Connect dialog.

KEY TAKEAWAY An Atlas cluster is reachable only after you allow an IP address and create a database user.

EXERCISE: IS THE SERVER RUNNING?

Read the evidence before you change anything.

PRACTICE EXERCISE

5 MIN

SCENARIO

A learner installed MongoDB on Windows. mongosh reports connection refused on localhost:27017.

```
PS> Get-Service MongoDB
```

Status	Name	DisplayName
-----	----	-----
Stopped	MongoDB	MongoDB Server (MongoDB)

YOUR TASKS

- 1 Name the layer that failed
- 2 Give the command that fixes it
- 3 Say how you'll prove the fix
- 4 Name a cause if it won't start

EXPECTED OUTCOME

- ✓ The server process: mongod is stopped
- ✓ Start-Service MongoDB, as Administrator
- ✓ mongosh, then db.runCommand({ ping: 1 })
- ✓ Check the log: bad dbPath or port in use

KEY TAKEAWAY Connection refused usually means nothing is listening — check the server before the client.

PART 3: CONNECTING TO MONGODB

Every client needs to know where the server is — and who is asking.



SO FAR

- ✓ mongod is installed and running
- ✓ Its config sets dbPath, log, port and bindIp
- ✓ Atlas needs a user and an IP allowlist

NEXT QUESTION

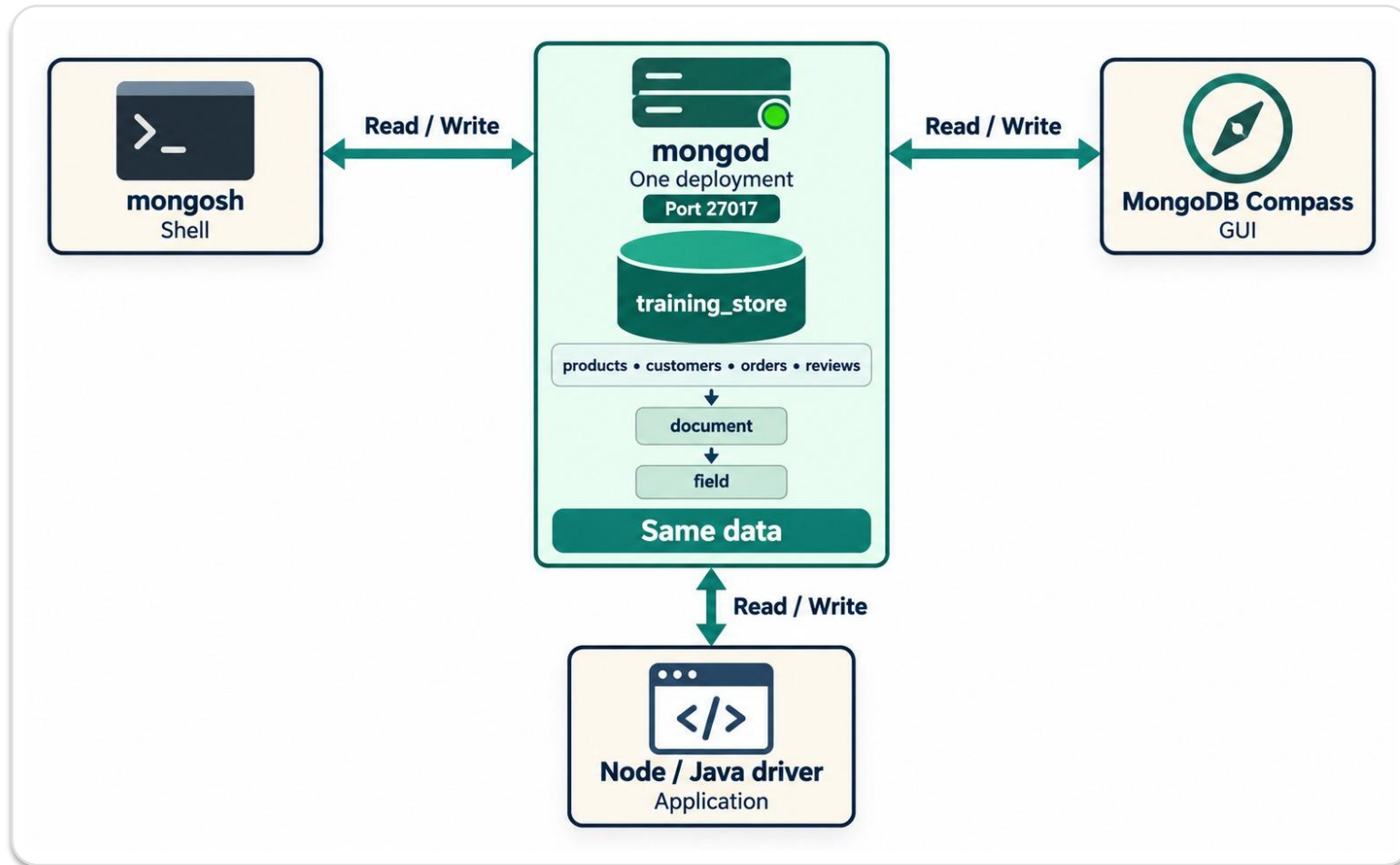
How does a client find the server, prove who it is, and confirm the connection works?

IN THIS PART

- 1 Three clients, one deployment
- 2 Localhost versus remote
- 3 Standard connection strings
- 4 SRV connection strings
- 5 Connecting with mongosh
- 6 Connecting with Compass
- 7 Verifying the connection

THREE CLIENTS, ONE DEPLOYMENT

mongosh, Compass and application drivers share the same data.



mongosh

The command-line shell for queries and administration.

MongoDB Compass

The graphical client to browse and edit documents.

Drivers

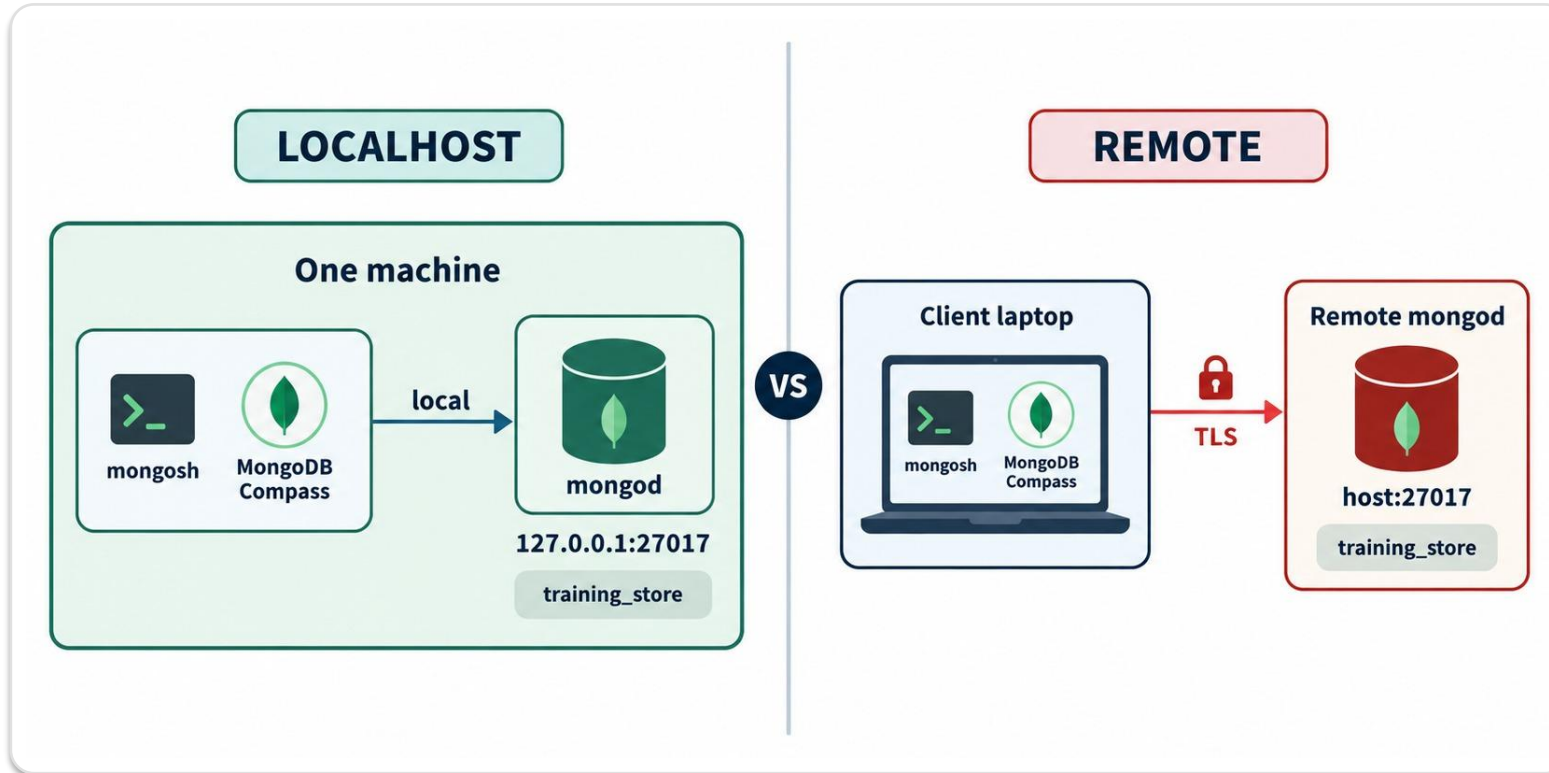
Libraries for application code — Node.js, Java and more.

SAME DATA A different window, not a different database.

KEY TAKEAWAY Every client reads and writes the same deployment — pick the one that suits the task.

LOCALHOST VS. REMOTE CONNECTIONS

One machine — or a client laptop and a remote server.



Localhost

127.0.0.1:27017 — traffic never leaves the machine.

Remote

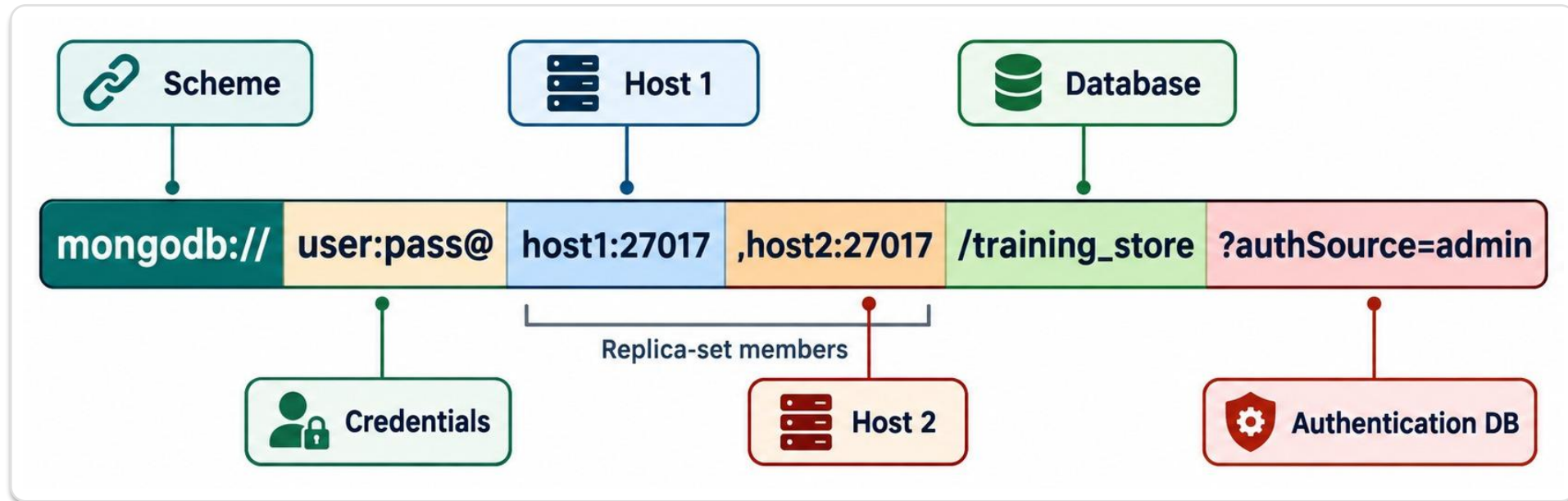
host:27017 across a network: TLS, authentication and an open firewall port.

BINDIP DECIDES mongod answers only on the addresses it binds to.

KEY TAKEAWAY Leaving localhost adds three requirements: a reachable port, TLS and authentication.

STANDARD CONNECTION-STRING FORMAT

Scheme, credentials, hosts, database and options.



Local examples

```
mongodb://localhost:27017
mongodb://localhost:27017/training_store
```

Read it left to right

Scheme, credentials, host:port list, database, then ?options.

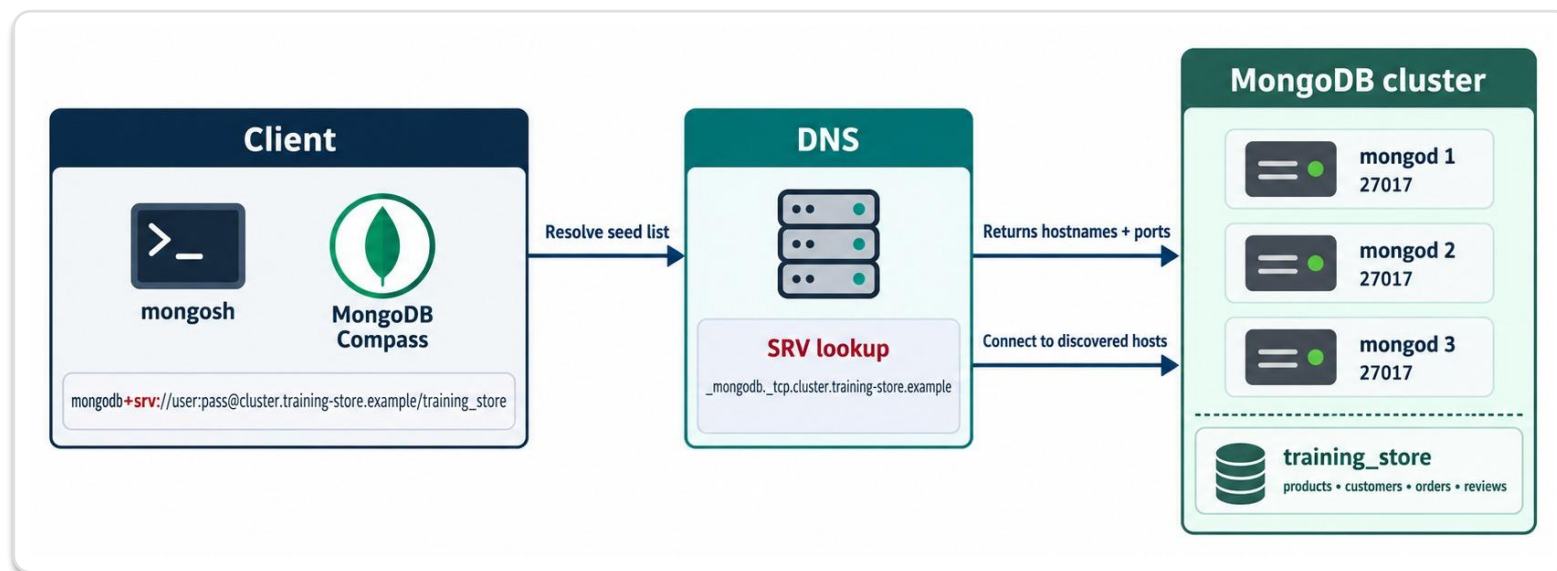
authSource and encoding

authSource names the user's database. Percent-encode special characters in passwords.

KEY TAKEAWAY A connection string answers five questions: how, who, where, which database and which options.

DNS SEED-LIST (SRV) CONNECTION STRINGS

mongodb+srv:// asks DNS for the member list.



Atlas pattern — placeholders only

```
mongodb+srv://  
  <username>:<password>@  
  <cluster-host>/
```

Seed list

DNS returns the members and ports — no port in the URI.

TLS on

SRV strings turn TLS on by default.

EXERCISE 2.2 Compare `mongodb://` and `mongodb+srv://`.

KEY TAKEAWAY `mongodb://` lists the hosts; `mongodb+srv://` lets DNS supply them.

CONNECTING WITH MONGOSH

Local defaults, a named database, and a password prompt.

Local server (PowerShell)

```
mongosh
# same as:
mongosh "mongodb://localhost:27017"

# open training_store directly:
mongosh "mongodb://localhost:27017/training_store"
```

✓ With a user: mongosh prompts for the password

```
mongosh "mongodb://localhost:27017/training_store" `
--username trainingUser

# Atlas: copy the host from Connect → Shell
mongosh "mongodb+srv://cluster0.example.mongodb.net/" `
--username trainingUser
Enter password: *****
```

mongosh on its own

Connects to localhost:27017 — the default.

--username

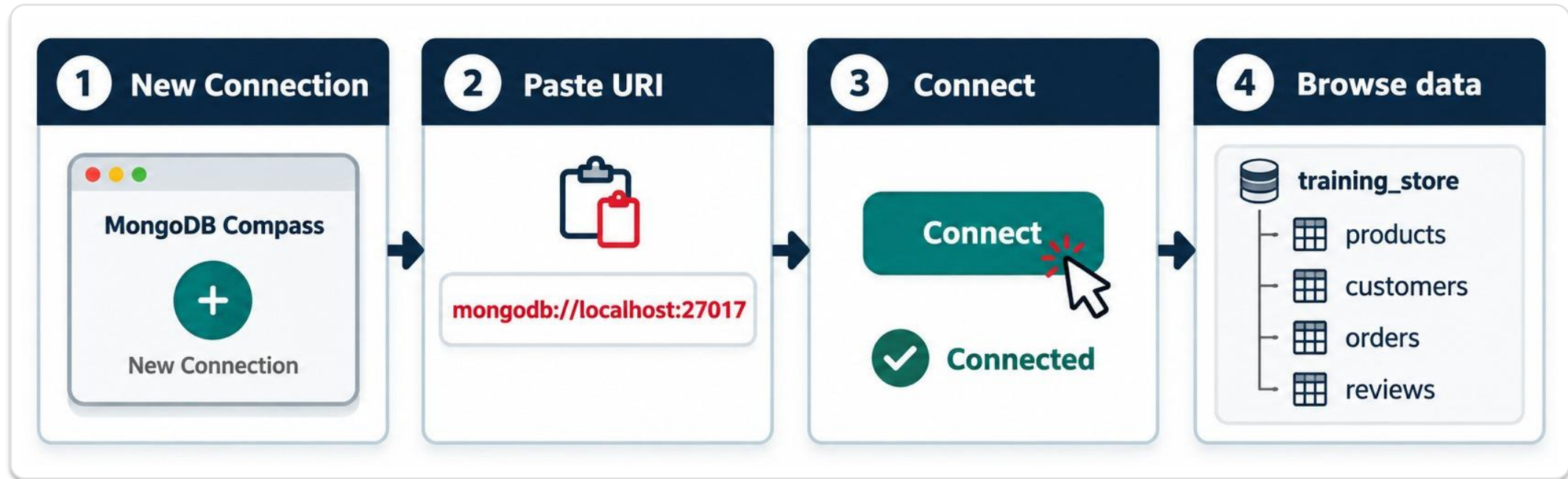
mongosh prompts for the password: nothing lands in history.

ATLAS Copy the exact command from Connect → Shell.

KEY TAKEAWAY Give mongosh a connection string and a username — and let it ask for the password.

CONNECTING WITH MONGODB COMPASS

New connection, paste the URI, connect, browse.



Same URI

Paste the connection string you used in mongosh.

Password field

Type it there — never in a shared screenshot.

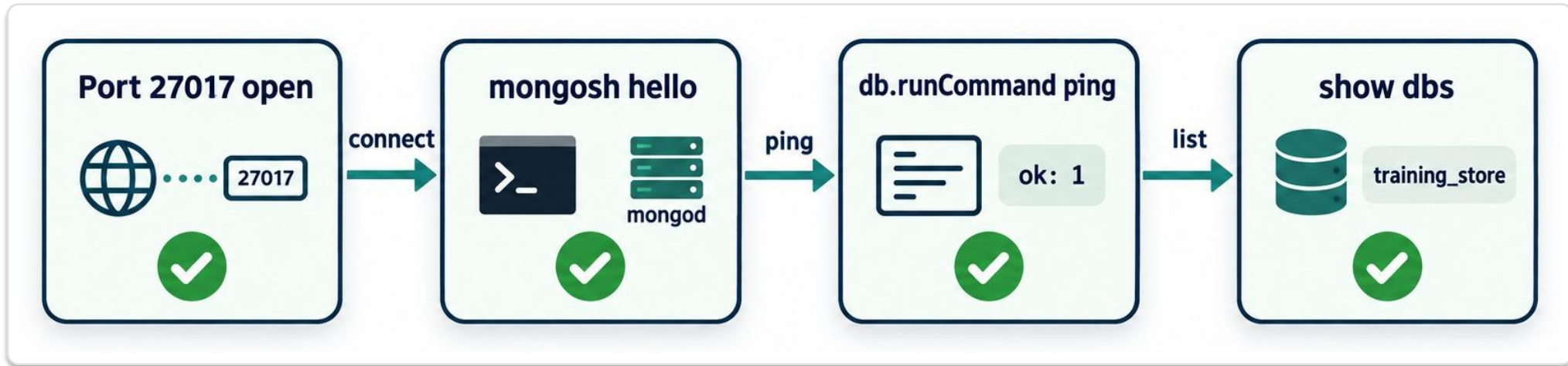
Browse

Databases → collections → documents; refresh to see new writes.

KEY TAKEAWAY Compass is a graphical window onto the same deployment, opened with the same connection string.

VERIFYING THE CONNECTION

Port open, shell connected, ping ok, databases listed.



Is the server answering?

```
test> db.runCommand({ ping: 1 })
{ ok: 1 }
test> db.version()
8.0.4
```

Where am I?

```
test> db
test
test> show dbs
admin    40.00 KiB
config  60.00 KiB
local   40.00 KiB
```

Reading the results

ok: 1 = reachable. A new server lists only admin, config and local; training_store joins after its first write.

KEY TAKEAWAY A ping that returns ok: 1 proves the server answers — the next part proves you can write.

EXERCISE: BUILD THE CONNECTION STRING

Write what each learner needs — with no password on the page.

PRACTICE EXERCISE

10 MIN

SCENARIO

Three learners need to connect. Write the connection string or mongosh command for each one.

- A Local server, default port
- B Local server, open training_store
- C Atlas: cluster0.example.mongodb.net as database user trainingUser

YOUR TASKS

- 1 Write learner A's URI
- 2 Add the database for learner B
- 3 Write learner C's mongosh command
- 4 Say where C's password goes

EXPECTED OUTCOME

- ✓ A: mongodb://localhost:27017
- ✓ B: add /training_store after the port
- ✓ C: the mongodb+srv:// URI plus -username
- ✓ Typed at the prompt — never in the URI

KEY TAKEAWAY If you can write the connection string, you can place most connection errors.

PART 4: FIRST COMMANDS ON TRAINING_STORE

Prove the environment: create, read, update and delete.



SO FAR

- ✓ Clients reach mongod through a connection string
- ✓ SRV strings let DNS find the cluster
- ✓ A ping proves the server answers

NEXT QUESTION

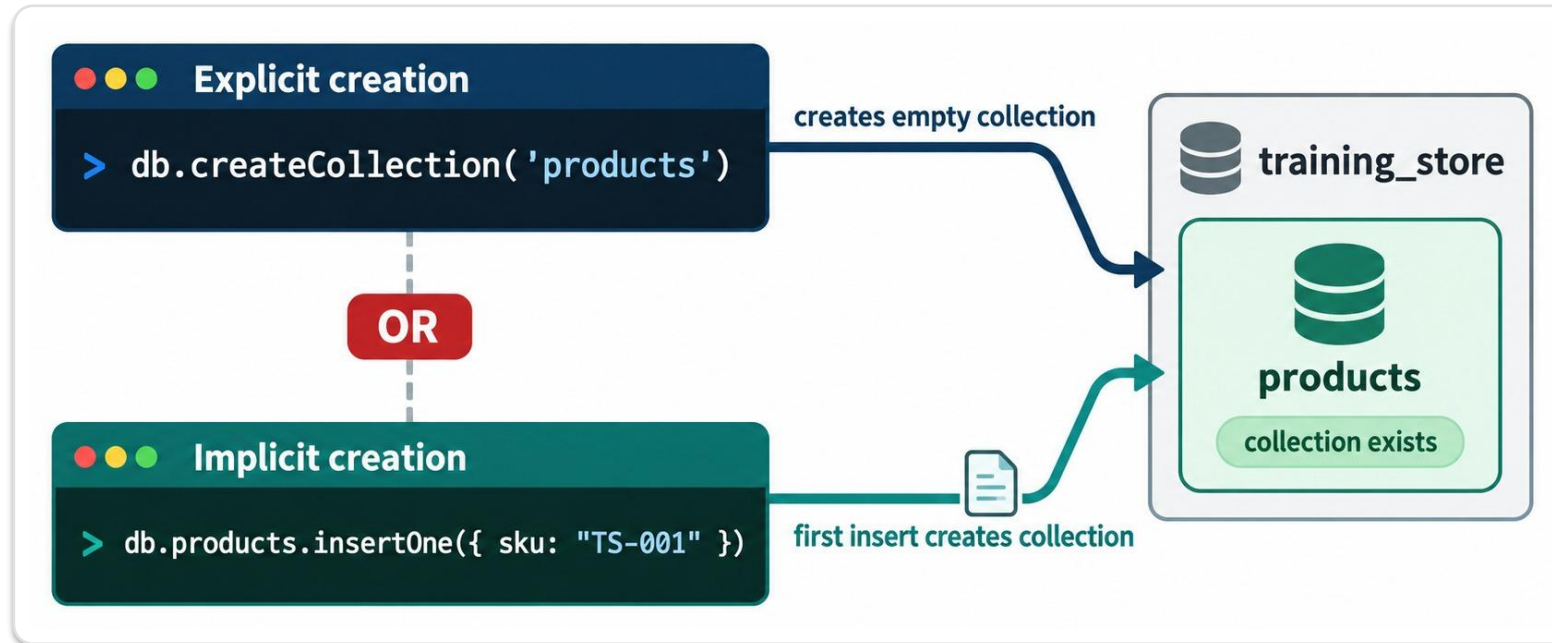
How do I create `training_store` — and prove I can write, read, update and delete?

IN THIS PART

- 1 Selecting a database and creating a collection
- 2 Inserting and reading a test document
- 3 Update and delete with a probe document
- 4 Loading the full `training_store` dataset
- 5 Predicting shell output

SELECTING A DATABASE AND CREATING A COLLECTION

use switches context; the first write creates the collection.



Explicit creation (Lab 1)

```
test> use training_store
switched to db training_store
training_store> db.createCollection(
  "products")
{ ok: 1 }
```

Implicit creation

The first insertOne creates the collection — no createCollection needed.

USE CREATES NOTHING show dbs lists training_store only after the first write.

KEY TAKEAWAY use only selects a name — a database and its collections appear with the first write.

INSERTING AND READING A TEST DOCUMENT

Lab 1's READY document in the environment_check collection.

Insert — MongoDB adds the `_id`

```
db.environment_check.insertOne({
  participant: "Student",
  environment: "training",
  shellConnected: true,
  status: "READY",
  checkedAt: new Date()
})
// result
{ acknowledged: true,
  insertedId: ObjectId('...') }
```

insertedId

An ObjectId, generated because we didn't supply an `_id`.

checkedAt

`new Date()` is stored as a BSON date; mongosh prints ISODate.

✓ Read it back

```
db.environment_check.findOne({
  status: "READY"
})
// result
{ _id: ObjectId('...'),
  participant: 'Student',
  environment: 'training',
  shellConnected: true,
  status: 'READY',
  checkedAt: ISODate('...') }
```

PROOF A write and a read = usable access.

KEY TAKEAWAY An acknowledged insert and a matching read prove more than any ping.

UPDATE, DELETE AND THE PROBE DOCUMENT

Day 1 Lab 1 proves all four CRUD operations with sku LAB1.

Create and read

```
db.products.insertOne({
  sku: "LAB1",
  name: "Connectivity Probe",
  category: "ACCESSORY",
  active: true
})
db.products.find({ sku: "LAB1" })
```

Create · Read

insertOne() returns an insertedId; find() shows the probe.

Update and delete

```
db.products.updateOne(
  { sku: "LAB1" },
  { $set: { verified: true } }
)
// matchedCount: 1, modifiedCount: 1
db.products.deleteOne({ sku: "LAB1" })
// { acknowledged: true, deletedCount: 1 }
db.products.find({ sku: "LAB1" })
// no result
```

Update · Delete

updateOne() matches 1 document; deleteOne() deletes 1.

REMOVE LAB1 Delete it before Lab 2 — it isn't a catalog SKU.

KEY TAKEAWAY insertOne, find, updateOne and deleteOne — four operations, four results you can check.

LOADING THE FULL TRAINING_STORE DATASET

Module 3's Lab 2 runs this — here is what to expect.

From the course folder, not inside mongosh

```
mongosh "mongodb://localhost:27017" `
  .\datasets\training_store\load.js

// prints
Loaded training_store
customers: 6
products: 13
orders: 17
reviews: 6
paid orders: 13
```

Drops, then reloads

customers, products, orders and reviews start clean each time.

Leaves the rest

environment_check isn't touched — it stays in the list.

✓ Then verify in mongosh

```
use training_store
show collections
// customers, environment_check,
// orders, products, reviews
db.products.countDocuments()
// 13
db.products.findOne(
  { sku: "B300" },
  { name: 1, price: 1, _id: 0 })
// { name: 'MongoDB Fundamentals',
//   price: Decimal128('59.99') }
```

MODULE 3 Lab 2 starts from exactly this state.

KEY TAKEAWAY One script gives everyone the same training_store — the starting point for every later lab.

EXERCISE: PREDICT THE SHELL OUTPUT

Say what mongosh prints — before you press Enter.

PRACTICE EXERCISE

5 MIN

SCENARIO

A brand-new local server. Nothing has been created yet.

```
show dbs
use training_store
show dbs
db.environment_check.insertOne(
  { status: "READY" })
show dbs
show collections
```

YOUR TASKS

- 1 Predict the first show dbs
- 2 Say what use prints — and creates
- 3 Predict show dbs after the insert
- 4 Predict show collections

EXPECTED OUTCOME

- ✓ admin, config and local only
- ✓ switched to db training_store; nothing on disk
- ✓ training_store now appears
- ✓ environment_check

KEY TAKEAWAY Databases and collections appear on the first write — predicting that saves a lot of confusion.

PART 5: SECURITY AND TROUBLESHOOTING

Keep credentials safe — and fix connections layer by layer.



SO FAR

- ✓ training_store exists on your server
- ✓ Insert, find, update and delete all work
- ✓ The full dataset loads with one script

NEXT QUESTION

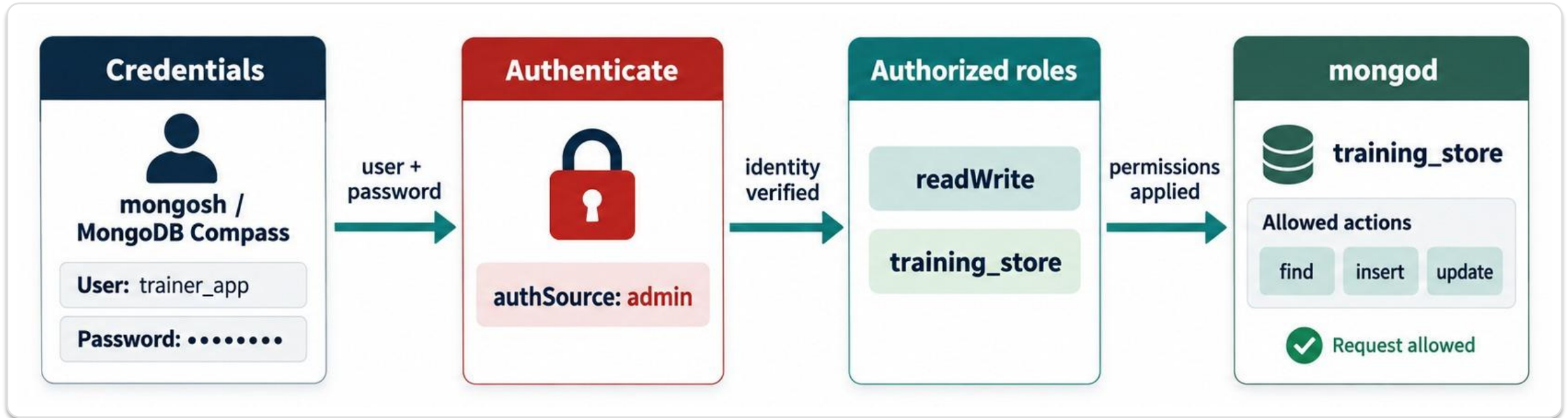
How do I keep credentials safe — and what do I check when a connection fails?

IN THIS PART

- 1 Authentication and authorization
- 2 Safe credential practices
- 3 Common connection problems
- 4 A troubleshooting workflow
- 5 Setup misconceptions

AUTHENTICATION AND AUTHORIZATION

Credentials, authentication, roles, allowed actions.



Authentication

Who are you? Username, password and authSource.

Authorization

What may you do? Roles such as readWrite.

Local default

Access control is off — tolerable on localhost only.

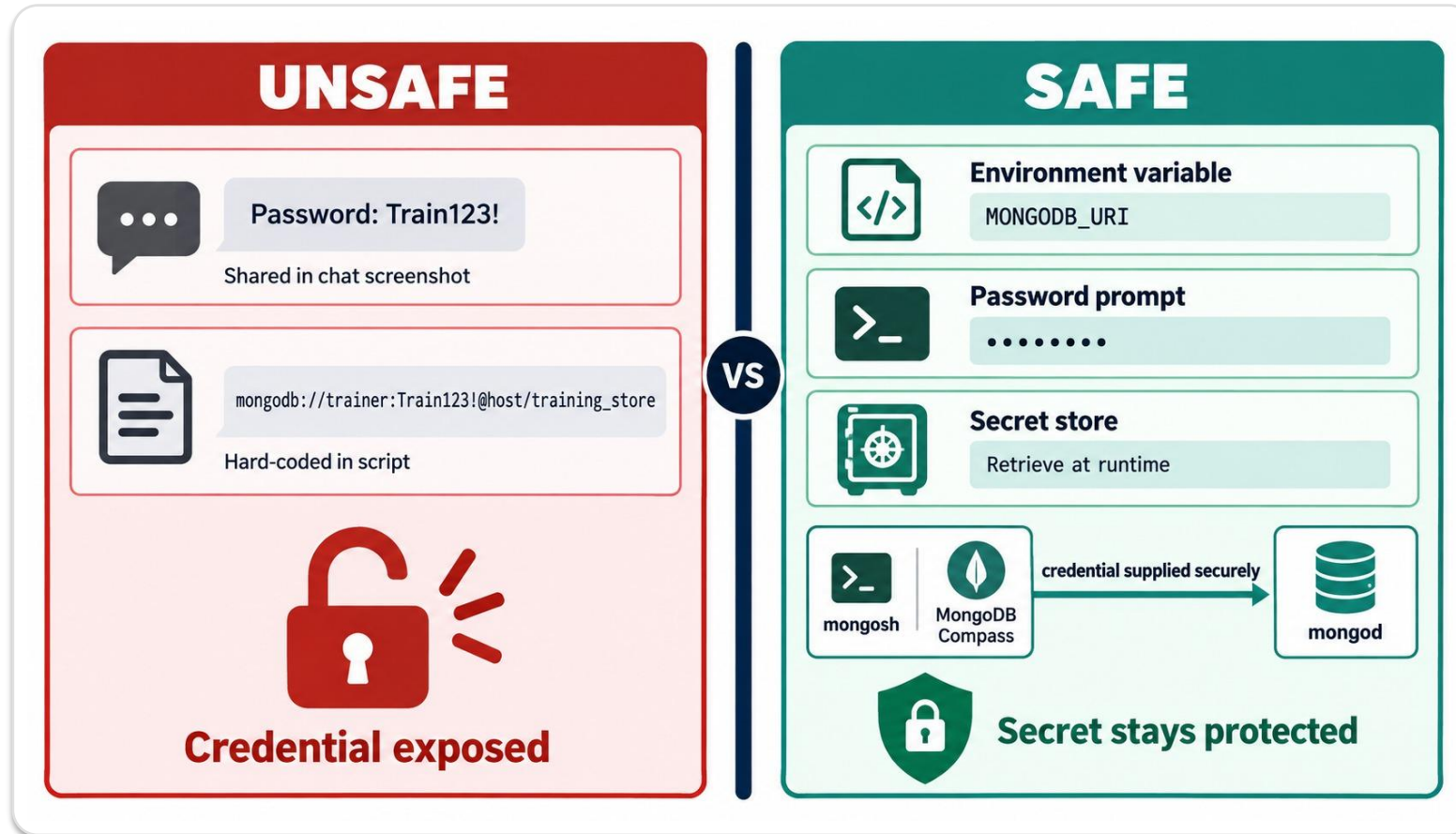
Least privilege

Give each user only the roles it needs.

KEY TAKEAWAY Authentication proves who you are; authorization decides what you may do.

SAFE CREDENTIAL PRACTICES

Keep passwords out of chat, screenshots, scripts and Git.



✓ Let mongosh prompt

```
# prompt instead of typing it
mongosh "<connection-string>" \
--username <username>
```

Store it safely

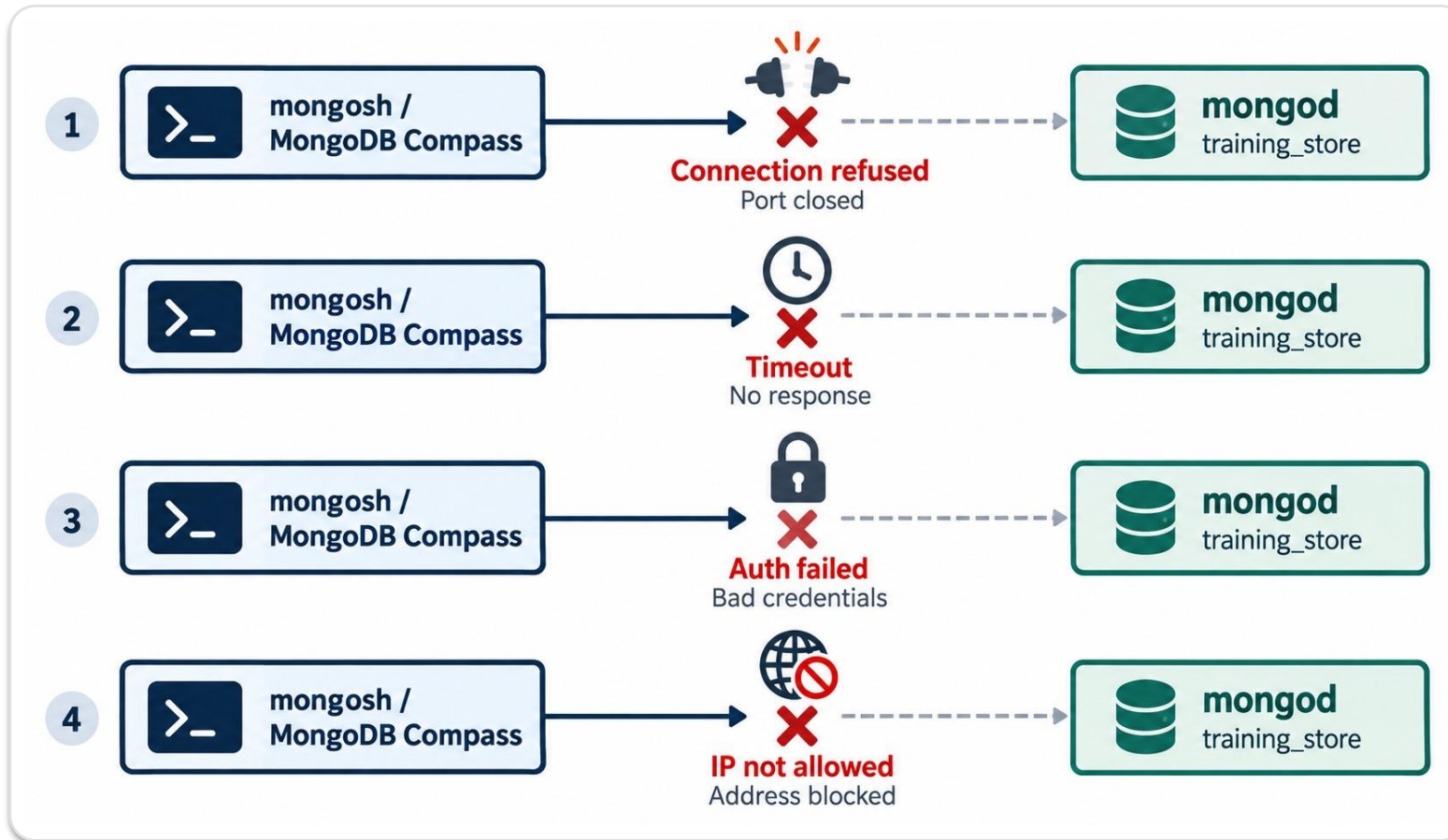
A password manager, a secret store or an environment variable.

0.0.0.0/0 Opens Atlas to the whole internet — avoid it; never in production.

KEY TAKEAWAY A password shown once must be treated as leaked — keep it out of every screen and file.

COMMON CONNECTION PROBLEMS

Refused, timed out, bad credentials, address blocked.



Connection refused

Nothing listening: mongod stopped, wrong port or bindIp.

Timeout

No answer: firewall, routing or the Atlas IP access list.

Auth failed

User, password, authSource or unencoded characters.

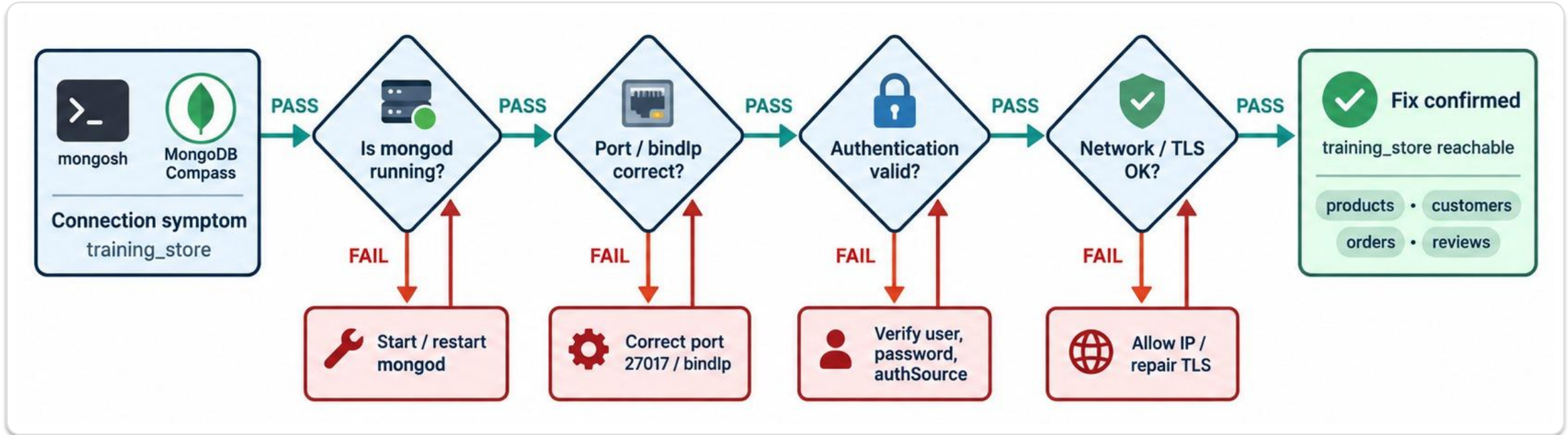
IP not allowed

Add your current IP — not 0.0.0.0/0.

KEY TAKEAWAY Each error names a different layer — read the message before changing anything.

A TROUBLESHOOTING WORKFLOW

Four checks, in order, until the fix is confirmed.



1 · Process

Is mongod running? Service status and log.

2 · Port and bindIp

Port 27017? Listening on your address?

3 · Credentials

User, password and authSource.

4 · Network and TLS

IP allowlist, firewall and TLS settings.

KEY TAKEAWAY Change one thing, retest with a ping, then move to the next layer.

SETUP MISCONCEPTIONS

Beliefs that waste the first hour of a lab.

MISCONCEPTION

REALITY

Closing mongosh stops MongoDB

mongod keeps running — it is a separate process

use training_store creates it

The database appears with the first write

A ping means I can write

Ping proves reachability — test a write and a read

Compass shows a different copy

Same deployment, same documents — refresh

0.0.0.0 makes connecting easier

It exposes the server — keep localhost or an allowlist

ANTI-PATTERNS TO AVOID

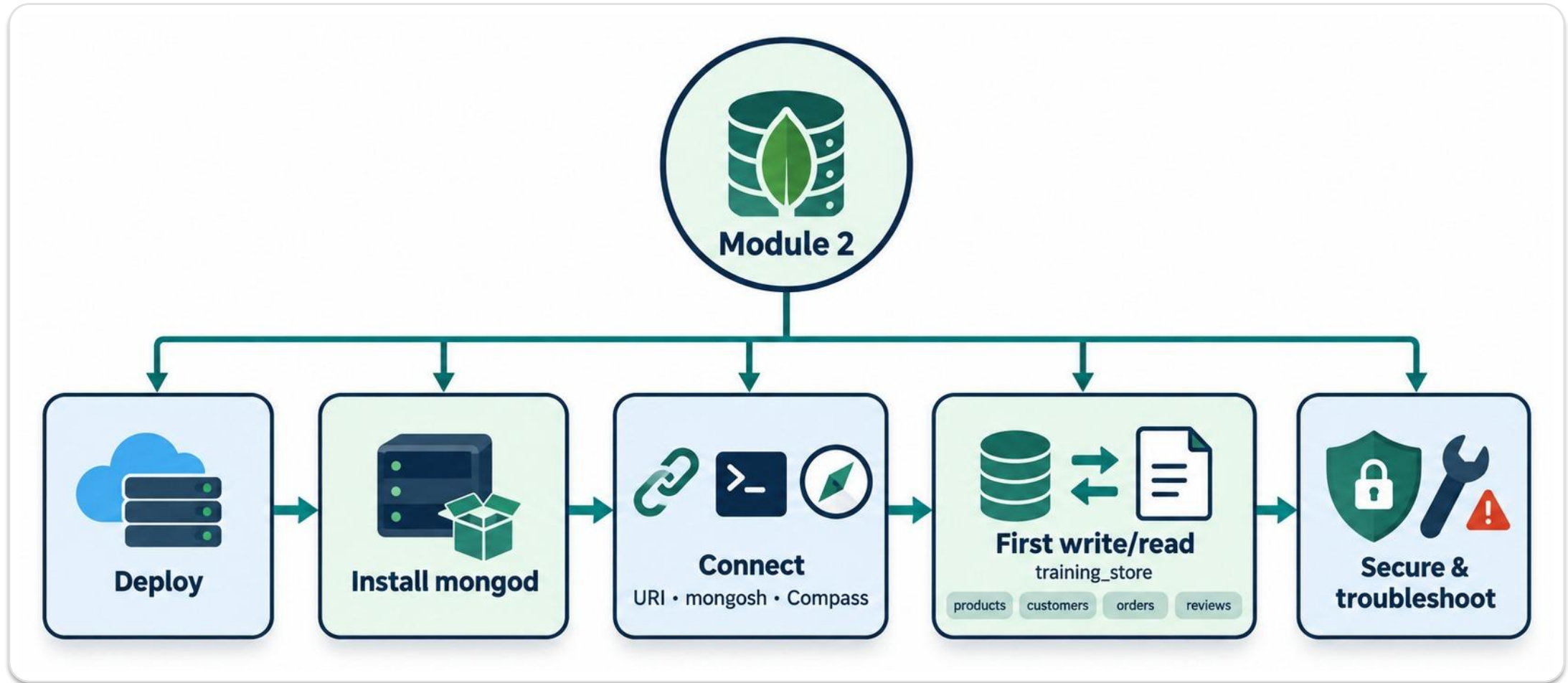
- ✗ Passwords in URIs, scripts or Git
- ✗ Screenshots of the connection dialog
- ✗ An Atlas login reused as a database user
- ✗ Changing three settings at once
- ✗ Killing mongod instead of stopping it

REMEMBER Prove each layer: running, reachable, authenticated, writable.

KEY TAKEAWAY Most setup problems come from confusing the client with the server, or reachability with access.

MODULE 2 AT A GLANCE

Deploy, install, connect, first write and read, secure and troubleshoot.



KEY TAKEAWAY A proven environment — running, reachable, authenticated and writable — is what every later lab needs.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is the role of mongod?

2 What is the role of mongosh?

3 What is MongoDB Compass?

4 What is MongoDB's default port?

5 How do mongodb:// and mongodb+srv:// differ?

6 What does db.runCommand({ ping: 1 }) verify?

7 Why might show dbs omit a database you just selected?

8 What commonly causes connection refused?

9 What commonly causes authentication failed?

10 Why not put credentials in source code?

EXERCISE 2.1 — SELECT A DEPLOYMENT OPTION

Module 2 checkpoint A: match six scenarios to a deployment, with a risk.

OFFICIAL CHECKPOINT A

10 MIN

labs/day-01/exercises/exercise-2.1-select-a-deployment-option.md

SCENARIO

Pick one option — local, self-managed server or VM, container, or managed cloud. Justify it and name the main risk.

- 1 Offline learning environment
- 2 Temporary automated tests
- 3 Distributed team, shared data
- 4 Regulated: direct control
- 5 Startup: no servers to manage
- 6 Classroom: restricted internet

YOUR TASKS

- 1 Name one good fit for each option
- 2 Classify scenarios 1–3
- 3 Classify scenarios 4–6
- 4 Defend one row against an alternative

SOLUTION (AFTER DEBRIEF)

- ✓ 1 local · 2 container
- ✓ 3 managed cloud
- ✓ 4 self-managed server or VM
- ✓ 5 managed cloud
- ✓ 6 local or a local server

KEY TAKEAWAY The commands barely change between options — the infrastructure responsibility does.

EXERCISE 2.2 — INTERPRET A CONNECTION STRING

Module 2 checkpoint B: label every part, and keep the password secret.

OFFICIAL CHECKPOINT B

10 MIN

labs/day-01/exercises/exercise-2.2-interpret-a-connection-string.md

SCENARIO

Label each part of this URI. It is one line, split to fit; the password is a placeholder.

```
mongodb://  
trainingUser:<password>@  
db.example.net:27017  
/training
```

YOUR TASKS

- 1 Label protocol, user, password, host, port, database
- 2 Answer the four security questions
- 3 Contrast mongodb:// with mongodb+srv://
- 4 Check labels without reading secrets aloud

SOLUTION (AFTER DEBRIEF)

- ✓ mongodb:// · trainingUser
- ✓ Password: placeholder only
- ✓ db.example.net · 27017 · training
- ✓ Secrets: env variable or secret store
- ✓ +srv: DNS discovers the hosts

KEY TAKEAWAY Parse a URI aloud — without ever reading the password onto a shared screen.

EXERCISE 2.3 — DIAGNOSE CONNECTION FAILURES

Module 2 checkpoint C: six failures, one layer at a time.

OFFICIAL CHECKPOINT C

15 MIN

labs/day-01/exercises/exercise-2.3-diagnose-connection-failures.md

SCENARIO

For each case: likely cause, first check, log or setting, fix and the command that verifies it.

- 1 ECONNREFUSED
- 2 Authentication failed
- 3 Server-selection timeout
- 4 DNS lookup failure
- 5 Unauthorized command
- 6 Invalid URI format

YOUR TASKS

- 1 Diagnose cases 1 and 2
- 2 Diagnose cases 3 and 4
- 3 Diagnose cases 5 and 6
- 4 Share one case: change one thing, retest

SOLUTION (AFTER DEBRIEF)

- ✓ 1 server stopped or wrong port
- ✓ 2 user, password or authSource
- ✓ 3 firewall, routing or allowlist
- ✓ 4 hostname or DNS
- ✓ 5 role too weak · 6 syntax or encoding
- ✓ Verify with a ping: ok: 1

KEY TAKEAWAY Timeout is not refused, and unauthorized is not unauthenticated — name the layer first.

EXERCISE 2.4 — ENVIRONMENT READINESS CHECKLIST

Module 2 checkpoint D: tick only what you have seen.

OFFICIAL CHECKPOINT D

10 MIN

labs/day-01/exercises/exercise-2.4-environment-readiness-checklist.md

SCENARIO

Fill it in during or right after Lab 1, the next slide. No passwords on the page.

- ☐ MongoDB installed or provisioned
- ☐ Server or cluster available
- ☐ mongosh and Compass available
- ☐ URI stored securely
- ☐ Network access configured
- ☐ Ping returns ok: 1
- ☐ Test insert and test read

YOUR TASKS

- 1 Record your path, host and port
- 2 Confirm mongosh --version and Compass
- 3 Tick ping, insert and read — if seen
- 4 List each gap: symptom, layer, next step

EXPECTED OUTCOME

- ✓ Path recorded, no secrets
- ✓ Only observed boxes ticked
- ✓ Every gap has a next step
- ✓ Ready for Module 3

KEY TAKEAWAY An honest checklist is worth more than a full one — never tick a box you didn't prove.

LAB 1 — INSTALL, CONNECT AND VERIFY

Day 1 hands-on: provision MongoDB, connect, and prove write and read.

OFFICIAL LAB

40 MIN

labs/day-01/lab1/LAB-1-GUIDE.md

SCENARIO

One path: A local Community Edition, B managed cloud, C an instructor URI. The mongosh steps are the same.

```
db.runCommand({ ping: 1 })
use training_store
db.createCollection(
  "environment_check")
db.environment_check.insertOne(
  { participant: "Student",
    status: "READY",
    checkedAt: new Date() })
db.environment_check.findOne(
  { status: "READY" })
```

YOUR TASKS

- 1 Install or provision on your path
- 2 Connect with mongosh and ping
- 3 Create environment_check; insert and find
- 4 insertMany, count, update your name
- 5 LAB1 probe: insert, find, update, delete
- 6 Compass, then the COMPLETE document

EXPECTED OUTCOME

- ✓ Ping returns ok: 1
- ✓ Insert acknowledged, _id assigned
- ✓ Count is 4; your name shows
- ✓ LAB1 deleted: find returns nothing
- ✓ Compass shows the same document
- ✓ No password shared anywhere

KEY TAKEAWAY The lab is finished when the Exercise 2.4 checklist matches what you actually saw.

MODULE 2 SUMMARY AND WHAT'S NEXT

From choosing a deployment to a proven environment.

YOU CAN NOW

- ✓ Choose a deployment option and justify it
- ✓ Explain mongod, mongosh, Compass and drivers
- ✓ Install, configure, start and stop MongoDB
- ✓ Read and build connection strings
- ✓ Create training_store and prove CRUD
- ✓ Protect credentials; troubleshoot by layer

NEXT: MODULE 3 — DATA MODELING WITH MONGODB

- 1 How do I load and inspect training_store?
- 2 When do I embed, and when do I reference?
- 3 How do BSON types and validation help?

BRING ALONG Your working connection, your Exercise 2.4 checklist and your Exercise 2.1 table.

KEY TAKEAWAY Your environment is proven — Module 3 loads training_store and starts designing documents.